13 DTOs IN NESTJS (DATA TRANSFER OBJECTS)

A DTO is just a plain class (or interface) that describes the SHAPE of data moving between the client and your server — usually the body of a POST/PUT request. It carries no logic of its own; it exists purely to say "this is what a valid request looks like."

```
Request body (raw JSON) -> DTO class -> handler method
                           (shape + validation contract)
```

WHY DTOs EXIST — THE PROBLEM THEY SOLVE

Right now, in this project, @Body() is read field-by-field:

```
// user.controller.ts (current code)
@Post('products')
addProduct(@Body('name') name: string, @Body('price') price: number) {
  return this.userService.addProduct(name, price);
}

@Put('products/:id')
updateProduct(
  @Param('id', ParseIntPipe) id: number,
  @Body('name') name: string,
  @Body('price') price: number,
) {
  return this.userService.updateProduct(id, name, price);
}
```

This works, but it has real problems:

1. NO RUNTIME VALIDATION — `price: number` is a TypeScript label only. If the client sends {"name": "Pen", "price": "free"}, `price` is really the string "free" at runtime; TypeScript can't stop that, only a compiler-time lie.
2. REPETITIVE — every field needs its own @Body('field') line, and the list grows every time the request body grows.
3. NO SINGLE SOURCE OF TRUTH — the "shape of a product" isn't written down anywhere; you'd have to read the handler signature to infer it.
4. HARD TO REUSE — the same shape (name + price) can't be shared between create, update, and the service layer without retyping it.

A DTO fixes all four: one class, one @Body() decorator, one visible contract.

THE SAME EXAMPLE, WITH A DTO

```
// dto/create-product.dto.ts
export class CreateProductDto {
  name: string;
  price: number;
}

// user.controller.ts
```

```
@Post('products')
addProduct(@Body() dto: CreateProductDto) {
  return this.userService.addProduct(dto.name, dto.price);
}
```

- @Body() with NO argument means "give me the whole request body," typed as CreateProductDto instead of `any`.
- One line covers the entire payload, no matter how many fields it has.

DTOs ALONE ARE ONLY A TYPE — VALIDATION NEEDS class-validator

By itself, the DTO above still doesn't enforce anything at runtime — it's the same TypeScript-only guarantee as before, just tidier. To actually reject bad requests, decorate the DTO with class-validator rules and run it through Nest's ValidationPipe (see theory/09-pipes.txt).

> NOTE: this project's package.json does not have class-validator or class-transformer installed yet. To use the pattern below: npm install class-validator class-transformer

```
// dto/create-product.dto.ts
import { IsString, IsNumber, Min } from 'class-validator';

export class CreateProductDto {
  @IsString()
  name: string;

  @IsNumber()
  @Min(0)
  price: number;
}

// main.ts — enable validation globally, once
app.useGlobalPipes(new ValidationPipe());
```

Now, POST /user/products with {"name": "Pen", "price": "free"} is rejected automatically with a 400 Bad Request before `addProduct` ever runs — no manual `if (typeof price !== 'number')` checks needed.

UPDATE DTOs — DON'T RETYPE THE SAME FIELDS

An update usually allows a SUBSET of the create fields (e.g. you can patch just the price). Nest's @nestjs/mapped-types gives PartialType to derive that automatically instead of duplicating the class:

```
// dto/update-product.dto.ts
import { PartialType } from '@nestjs/mapped-types';
import { CreateProductDto } from './create-product.dto';

export class UpdateProductDto extends PartialType(CreateProductDto) {}
// -> { name?: string; price?: number }  (all fields now optional)

@Put('products/:id')
updateProduct(@Param('id', ParseIntPipe) id: number, @Body() dto: UpdateProductDto) {
  return this.userService.updateProduct(id, dto.name, dto.price);
}
```

WHERE DTO FILES LIVE (CONVENTION)

Nest projects typically keep them next to the feature module:

```
src/user/dto/create-product.dto.ts
src/user/dto/update-product.dto.ts
```

Naming convention: <Action><Entity>Dto, e.g. CreateProductDto, UpdateProductDto, CreateUserDto.

DTO vs INTERFACE vs ENTITY (QUICK DISTINCTION)

```
DTO        – shape of data crossing the network boundary (request/
             response). Can carry validation decorators because it's
             a real class, not just a compile-time type.
Interface  – pure TypeScript compile-time shape, erased at runtime.
             Can't be decorated with class-validator (decorators need
             a real class to attach to), so DTOs are written as
             classes even though an interface looks simpler.
Entity     – shape of a row/document in the database (e.g. a
             TypeORM/Mongoose model). Often similar to a DTO but
             represents STORED data, not necessarily what the client
             is allowed to send (e.g. an entity has `id`, `createdAt`;
             a CreateDto usually doesn't).
```

★ RULE OF THUMB

Any time a handler reads more than one field off @Body(), or the same shape needs to be reused across create/update/service, reach for a DTO class instead of listing @Body('field') params one by one. Pair it with class-validator + ValidationPipe when the data must be *trusted*, not just typed.

See also: theory/09-pipes.txt, theory/02-controller.txt